This file is a merged representation of the entire codebase, combined into a single document by Repomix.

File Summary

Purpose

This file contains a packed representation of the entire repository’s contents. It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

File Format

The content is organized as follows: 1. This summary section 2. Repository information 3. Directory structure 4. Repository files (if enabled) 5. Multiple file entries, each consisting of: a. A header with the file path (## File: path/to/file) b. The full contents of the file in a code block

Usage Guidelines

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

Notes

- Some files may have been excluded based on .gitignore rules and Repomix’s configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

Directory Structure

```
BarWidget.qml
gen-pdf.sh
manifest.json
output.pdf
workspace.py
```

Files

File: BarWidget.qml

```
import QtQuick
import Quickshell
import Quickshell.Io
import qs.Commons
import qs.Ui

BarWidget {
  id: root
  moduleName: "kai.git-status"

  // Существующие свойства плагина
  property var repoRows: []
  property int dirtyCount: 0
  property int untrackedCount: 0
  property int missingCount: 0
  property int errorCount: 0
  property string statusSummary: ""
  property string statusError: ""
  property bool panelOpen: false
  property bool busy: false
  property string panelAction: "status"
  property string panelSummary: ""
  property string panelError: ""
  property var panelRows: []
  property bool vpnActive: false
  property string pendingAction: ""
  property string pendingArg: ""
  property bool vpnConfirmOpen: false
  property string vpnPhase: ""
  property string busyText: "Working…"
```

```
  readonly property bool opened: panelOpen

  // Подпись текущей вкладки / выполняемого действия для заголовка
панели
  readonly property string headerState: {
    if (busy) {
      if (vpnPhase !== "") return "VPN…"
      if (branchProcess.running) return "Switching…"
      if (panelAction === "pull") return "Pulling…"
      if (panelAction === "push") return "Pushing…"
      if (panelAction === "fetch") return "Fetching…"
      if (panelAction === "status") return "Checking…"
      if (panelAction === "branches") return "Loading…"
      return "Working…"
    }
    if (panelAction === "branches") return "Branches"
    if (panelAction === "pull") return "Pull"
    if (panelAction === "push") return "Push"
    if (panelAction === "fetch") return "Fetch"
    if (panelAction === "status") return "Status"
    return ""
  }
  readonly property string home: Quickshell.env("HOME") || ""
  readonly property string icon: "\uf1d3"
  readonly property string vpnConnection:
String(setting("vpnConnection", "FALT")).trim()
  readonly property string vpnMode: {
    var mode = String(setting("vpnMode",
"auto")).trim().toLowerCase()
    return mode === "always" || mode === "never" ? mode : "auto"
  }
  readonly property string workspacePath:
expandPath(String(setting("workspacePath", "~/BAS/bcad-ws")))
  readonly property string scriptPath:
decodeURIComponent(String(Qt.resolvedUrl("workspace.py")).replace(/
^file:\/\//, ""))
  readonly property string packagesPath: workspacePath + "/
packages.json"
  readonly property bool showWidget: repoRows.length > 0 ||
statusError !== ""

  // Свойства для управления новым оверлеем веток
  property bool branchOverlayOpen: false
  property var selectedRepoData: null
  property string newBranchName: ""
  property string overlayMessage: ""
```

```
  property string overlayError: ""

  readonly property string barText: {
    if (root.vertical) return icon
      if (statusError !== "") return icon + " !"
        if (dirtyCount > 0) return icon + " " + dirtyCount +
(untrackedCount > 0 ? "?" : "")
          if (untrackedCount > 0) return icon + " ?"
            if (missingCount > 0 || errorCount > 0) return icon +
" !"
              return icon
  }
  readonly property string tooltip: buildTooltip()

  function expandPath(value) {
    if (value === "~") return home
      if (value.indexOf("~/") === 0) return home +
value.substring(1)
        return value
  }

  function refresh() {
    if (statusProcess.running) return
      statusProcess.command = ["python3", root.scriptPath,
"status", root.workspacePath]
      statusProcess.running = true
  }

  function open() { panelOpen = true }
  function close() {
    if (vpnConfirmOpen) vpnConfirmOpen = false
      panelOpen = false
      branchOverlayOpen = false
  }
  function togglePanel() { if (panelOpen) close(); else panelOpen
= true }

  function actionLabel(action) {
    if (action === "pull") return "Pull"
      if (action === "push") return "Push"
        if (action === "fetch") return "Fetch"
          if (action === "status") return "Status"
            if (action === "branches") return "Branches"
              return action
  }
```

```
  function runAction(action, arg) {
    if (branchProcess.running || actionProcess.running ||
vpnCheckProcess.running || vpnUpProcess.running) return
      panelAction = action
      panelError = ""
      busy = true
      busyText = "Running " + actionLabel(action) + "…"
      actionProcess.command = ["python3", root.scriptPath, action,
root.workspacePath].concat(arg ? [String(arg)] : [])
      actionProcess.running = true
  }

  function runFetch(dest) {
    overlayMessage = ""
    overlayError = ""
    requestNetworkAction("fetch", dest)
  }

  function switchBranch(dest, branch, create) {
    if (branchProcess.running || actionProcess.running ||
vpnCheckProcess.running || vpnUpProcess.running) return
      var name = String(branch || "").trim()
      if (name === "") {
        panelError = "Branch name is required"
        return
      }
      panelError = ""
      busy = true
      busyText = (create ? "Creating " : "Switching to ") + name +
"…"
      branchProcess.command = ["python3", root.scriptPath,
"switch-branch", root.workspacePath, dest, name, create ?
"create" : "switch"]
      branchProcess.running = true
      branchOverlayOpen = false // Закрываем оверлей после запуска
команды
  }

  function requestNetworkAction(action, arg) {
    if (branchProcess.running || actionProcess.running ||
vpnCheckProcess.running || vpnUpProcess.running) return
      if (vpnMode === "never" || vpnConnection === "") {
        runAction(action, arg)
        return
      }
      pendingAction = action
```

```
      pendingArg = String(arg || "")
      panelAction = action
      panelError = ""
      busy = true
      busyText = "Checking VPN " + vpnConnection + "…"
      // auto: сначала проверяем, что VPN-профиль вообще настроен
на этой машине
      // (на работе FALT нет — тогда действуем напрямую). always:
сразу проверяем активность.
      vpnPhase = vpnMode === "always" ? "precheck" :
"profilecheck"
      vpnCheckProcess.command = vpnMode === "always"
        ? ["nmcli", "-t", "-f", "NAME", "connection", "show", "--
active"]
        : ["nmcli", "-t", "-f", "NAME", "connection", "show"]
      vpnCheckProcess.running = true
  }

  function connectVpn() {
    if (vpnUpProcess.running) return
      vpnConfirmOpen = false
      panelError = ""
      busy = true
      busyText = "Connecting VPN " + vpnConnection + "…"
      vpnUpProcess.command = ["nmcli", "connection", "up",
vpnConnection]
      vpnUpProcess.running = true
  }

  function cancelVpnConfirm() { vpnConfirmOpen = false;
pendingAction = ""; pendingArg = ""; panelSummary = "Cancelled" }

  function applyStatus(data) {
    repoRows = Array.isArray(data.rows) ? data.rows : []
    dirtyCount = Number(data.dirtyCount) || 0
    untrackedCount = Number(data.untrackedCount) || 0
    missingCount = Number(data.missingCount) || 0
    errorCount = Number(data.errorCount) || 0
    statusSummary = String(data.summary || "")
    statusError = data.error ? String(data.error) : ""
    if (panelAction === "status" && panelRows.length > 0) {
      panelRows = repoRows
      panelSummary = statusSummary
      panelError = statusError
    }
  }
```

```
  function applyActionResult(data) {
    panelRows = Array.isArray(data.rows) ? data.rows : []
    panelSummary = String(data.summary || "")
    panelError = data.error ? String(data.error) : ""
    if (data.action === "status") applyStatus(data)
  }

  // Fetch внутри оверлея веток: обновляем список веток, оверлей
не закрываем
  function applyFetchResult(data) {
    if (branchOverlayOpen && selectedRepoData) {
      if (data.error) overlayError = String(data.error)
        else overlayMessage = String(data.summary || "")
      var rows = Array.isArray(data.rows) ? data.rows : []
      for (var i = 0; i < rows.length; i++) {
        if (rows[i].dest === selectedRepoData.dest) {
          selectedRepoData = rows[i]
          overlayBranchPicker.options = rows[i].branches || []
          overlayBranchPicker.value = rows[i].branch || ""
          break
        }
      }
    }
    refresh()
  }

  function parseOutput(text) {
    try { return JSON.parse(text) } catch (error) { return null }
  }

  function rowColor(state) {
    if (state === "clean" || state === "ok" || state ===
"untracked") return Color.accent
      if (state === "modified" || state === "mismatch" || state
=== "missing" || state === "error" || state === "failed") return
Color.urgent
        return Color.foreground
  }

  function rowDetailColor(state) {
    if (state === "untracked" || state === "mismatch") return
Color.muted
      return Color.urgent
  }
```

```
  function buildTooltip() {
    var lines = []
    if (statusError !== "") {
      lines.push("Workspace: " + statusError)
      lines.push(packagesPath)
      return lines.join("\n")
    }
    if (repoRows.length === 0) {
      lines.push("Workspace: no repositories")
      lines.push(packagesPath)
      return lines.join("\n")
    }
    lines.push("BCAD workspace" + (statusSummary !== "" ? " · " +
statusSummary : ""))
    var problems = []
    for (var i = 0; i < repoRows.length; i++) {
      var row = repoRows[i]
      if (row.state === "clean" || row.state === "ok") continue
        problems.push(row.dest + ": " + (row.label || row.state ||
""))
    }
    if (problems.length === 0) {
      lines.push("All clean")
    } else {
      for (var j = 0; j < problems.length; j++)
lines.push(problems[j])
    }
    return lines.join("\n")
  }

  onPanelOpenChanged: {
    if (panelOpen) {
      runAction("status")
    } else if (vpnConfirmOpen) {
      vpnConfirmOpen = false
      pendingAction = ""
    }
  }

  visible: showWidget
  implicitWidth: button.implicitWidth
  implicitHeight: button.implicitHeight

  // IPC Handler
  IpcHandler {
    target: "kai.git-status"
```

```
    function refresh(): void { root.broadcast("refresh") }
    function open(): void { root.open() }
    function close(): void { root.close() }
    function show(): void { root.open() }
    function hide(): void { root.close() }
    function toggle(): void { root.togglePanel() }
    function status(): void { root.runAction("status") }
    function branches(): void { root.runAction("branches") }
    function switchBranch(dest: string, branch: string, create:
bool): void { root.switchBranch(dest, branch, create) }
    function pull(): void { root.requestNetworkAction("pull") }
    function push(): void { root.requestNetworkAction("push") }
    function fetch(dest: string): void { root.runFetch(dest) }
    function cancelVpn(): void { root.cancelVpnConfirm() }
    function confirmVpn(): void { root.connectVpn() }
    function state(): string {
      return JSON.stringify({
        action: root.panelAction,
        headerState: root.headerState,
        workspacePath: root.workspacePath,
        panelOpen: root.panelOpen,
        busy: root.busy,
        vpnMode: root.vpnMode,
        vpnConnection: root.vpnConnection,
        vpnActive: root.vpnActive,
        vpnConfirmOpen: root.vpnConfirmOpen,
        pendingAction: root.pendingAction,
        summary: root.panelSummary,
        error: root.panelError,
        rows: root.panelRows,
        dirtyCount: root.dirtyCount,
        untrackedCount: root.untrackedCount,
        statusSummary: root.statusSummary,
        tooltip: root.tooltip
      })
    }
  }

  // Процессы бэкенда
  Process {
    id: statusProcess
    stdout: StdioCollector { id: statusStdout; waitForEnd: true }
    stderr: StdioCollector { id: statusStderr; waitForEnd: true }
    onExited: function(exitCode) {
      if (exitCode !== 0) {
        root.statusError = statusStderr.text.trim() ||
```

```
"workspace.py failed"
        return
      }
      var data = root.parseOutput(statusStdout.text)
      if (data) root.applyStatus(data)
        else root.statusError = "could not parse status output"
    }
  }

  Process {
    id: actionProcess
    stdout: StdioCollector { id: actionStdout; waitForEnd: true }
    stderr: StdioCollector { id: actionStderr; waitForEnd: true }
    onExited: function(exitCode) {
      root.busy = false
      var wasFetch = root.panelAction === "fetch"
      if (exitCode !== 0) {
        var failMessage = actionStderr.text.trim() ||
"workspace.py failed"
        root.panelError = failMessage
        if (wasFetch) root.overlayError = failMessage
        return
      }
      var data = root.parseOutput(actionStdout.text)
      if (!data) {
        root.panelError = "could not parse action output"
        if (wasFetch) root.overlayError = "could not parse fetch
output"
        return
      }
      root.applyActionResult(data)
      if (wasFetch) root.applyFetchResult(data)
    }
  }
  Process {
    id: branchProcess
    stdout: StdioCollector { id: branchStdout; waitForEnd: true }
    stderr: StdioCollector { id: branchStderr; waitForEnd: true }
    onExited: function(exitCode) {
      root.busy = false
      if (exitCode !== 0) {
        root.panelError = branchStderr.text.trim() ||
"workspace.py failed"
        return
      }
      var data = root.parseOutput(branchStdout.text)
```

```
      if (!data) {
        root.panelError = "could not parse switch-branch output"
        return
      }
      root.applyActionResult(data)
      if (!data.error) root.refresh()
    }
  }
  Process {
    id: vpnCheckProcess
    stdout: StdioCollector { id: vpnCheckStdout; waitForEnd:
true }
    stderr: StdioCollector { id: vpnCheckStderr; waitForEnd:
true }
    function nameListed(text, name) {
      var lines = String(text).split("\n")
      for (var i = 0; i < lines.length; i++) {
        if (lines[i].trim() === name) return true
      }
      return false
    }
    onExited: function(exitCode) {
      var phase = root.vpnPhase
      root.vpnPhase = ""

      if (phase === "profilecheck") {
        if (exitCode !== 0) {
          root.busy = false
          root.pendingAction = ""
          root.pendingArg = ""
          root.panelError = vpnCheckStderr.text.trim() || "could
not read NetworkManager connections"
          return
        }
        if (!nameListed(vpnCheckStdout.text, root.vpnConnection))
{
          // Профиля VPN на этой машине нет (рабочий вариант) —
запускаем действие напрямую
          var directAction = root.pendingAction
          var directArg = root.pendingArg
          root.pendingAction = ""
          root.pendingArg = ""
          root.busy = false
          root.runAction(directAction, directArg)
          return
        }
```

```
        root.vpnPhase = "precheck"
        root.busyText = "Checking VPN " + root.vpnConnection + "…"
        vpnCheckProcess.command = ["nmcli", "-t", "-f", "NAME",
"connection", "show", "--active"]
        vpnCheckProcess.running = true
        return
      }

      var active = exitCode === 0 &&
nameListed(vpnCheckStdout.text, root.vpnConnection)
      root.vpnActive = active
      if (phase === "precheck" && exitCode !== 0) {
        root.busy = false
        root.pendingAction = ""
        root.pendingArg = ""
        root.panelError = vpnCheckStderr.text.trim() || "could not
check VPN " + root.vpnConnection
        return
      }
      if (phase === "precheck" && !active) {
        root.busy = false
        if (root.panelOpen) root.vpnConfirmOpen = true
          else {
            root.pendingAction = ""
            root.pendingArg = ""
          }
            return
      }
      if (phase === "postcheck" && !active) {
        root.busy = false
        root.pendingAction = ""
        root.pendingArg = ""
        root.panelError = "VPN " + root.vpnConnection + " is not
active"
        return
      }
      if (phase === "precheck" || phase === "postcheck") {
        var action = root.pendingAction
        var arg = root.pendingArg
        root.pendingAction = ""
        root.pendingArg = ""
        root.busy = false
        root.runAction(action, arg)
      }
    }
  }
```

```
  Process {
    id: vpnUpProcess
    stdout: StdioCollector { id: vpnUpStdout; waitForEnd: true }
    stderr: StdioCollector { id: vpnUpStderr; waitForEnd: true }
    onExited: function(exitCode) {
      if (exitCode !== 0) {
        root.busy = false
        root.pendingAction = ""
        root.pendingArg = ""
        root.panelError = vpnUpStderr.text.trim() || "could not
connect VPN " + root.vpnConnection
        return
      }
      root.vpnPhase = "postcheck"
      root.busyText = "Waiting for VPN " + root.vpnConnection +
"…"
      vpnCheckProcess.command = ["nmcli", "-t", "-f", "NAME",
"connection", "show", "--active"]
      vpnCheckProcess.running = true
    }
  }
  Timer {
    interval: 30000
    running: true
    repeat: true
    triggeredOnStart: true
    onTriggered: root.refresh()
  }
  WidgetButton {
    id: button
    anchors.fill: parent
    bar: root.bar
    text: root.barText
    tooltipText: root.tooltip
    active: root.dirtyCount > 0 || root.untrackedCount > 0 ||
root.missingCount > 0 || root.errorCount > 0 || root.statusError !
== ""
    fontSize: Style.font.body
    onPressed: function(b) {
      if (b === Qt.RightButton) root.refresh()
        else root.togglePanel()
    }
  }
  KeyboardPanel {
    id: popup
    anchorItem: button       // Привязка к кнопке на панели
```

```
    owner: root
    bar: root.bar
    open: root.panelOpen     // Синхронизация состояния открытия

    contentWidth: popup.fittedContentWidth(Style.space(480))
    contentHeight:
popup.fittedContentHeight(panelColumn.implicitHeight,
Style.space(620))

    Column {
      id: panelColumn
      anchors.fill: parent
      spacing: Style.space(8)
      Row {
        width: parent.width
        spacing: Style.space(6)
        Text {
          width: parent.width - headerStateText.width -
parent.spacing
          text: "BCAD workspace"
          color: Color.foreground
          font.family: Style.font.family
          font.pixelSize: Style.font.body
          font.bold: true
          elide: Text.ElideRight
        }
        Text {
          id: headerStateText
          anchors.verticalCenter: parent.verticalCenter
          width: implicitWidth // пустой текст даёт нулевую ширину
без отдельной видимости
          text: root.headerState
          color: root.busy ? Color.accent : Color.muted
          font.family: Style.font.family
          font.pixelSize: Style.font.caption
        }
      }
      Text {
        width: parent.width
        text: root.workspacePath
        color: Qt.darker(Color.foreground, 1.5)
        font.family: Style.font.family
        font.pixelSize: Style.font.caption
        elide: Text.ElideMiddle
      }
      Row {
```

```
        width: parent.width
        spacing: Style.space(6)
        Button {
          text: "Status"
          bordered: true
          fontSize: Style.font.caption
          opacity: root.busy ? 0.5 : 1
          onClicked: if (!root.busy) root.runAction("status")
        }
        Button {
          text: "Branches"
          bordered: true
          fontSize: Style.font.caption
          opacity: root.busy ? 0.5 : 1
          onClicked: if (!root.busy) root.runAction("branches")
        }
        Button {
          text: "Pull"
          bordered: true
          fontSize: Style.font.caption
          opacity: root.busy ? 0.5 : 1
          onClicked: if (!root.busy)
root.requestNetworkAction("pull")
        }
        Button {
          text: "Push"
          bordered: true
          fontSize: Style.font.caption
          opacity: root.busy ? 0.5 : 1
          onClicked: if (!root.busy)
root.requestNetworkAction("push")
        }
      }
      Rectangle {
        width: parent.width
        height: 1
        color: Color.foreground
        opacity: 0.18
      }
      Text {
        width: parent.width
        visible: root.busy || root.panelError !== "" ||
root.panelSummary !== ""
        text: root.busy ? root.busyText : (root.panelError !==
"" ? root.panelError : root.panelSummary)
        color: root.panelError !== "" ? Color.urgent :
```

```
Qt.darker(Color.foreground, 1.35)
        font.family: Style.font.family
        font.pixelSize: Style.font.caption
        wrapMode: Text.Wrap
      }
      Text {
        width: parent.width
        visible: root.panelRows.length === 0 && !root.busy &&
root.panelError === ""
        text: "Choose an action"
        color: Qt.darker(Color.foreground, 1.6)
        font.family: Style.font.family
        font.pixelSize: Style.font.caption
        horizontalAlignment: Text.AlignHCenter
      }
      Flickable {
        width: parent.width
        height: root.panelRows.length === 0 ? 0 :
Math.max(Style.space(28), Math.min(rowColumn.implicitHeight,
Style.space(380)))
        visible: root.panelRows.length > 0
        contentWidth: width
        contentHeight: rowColumn.implicitHeight
        clip: true
        boundsBehavior: Flickable.StopAtBounds
        Column {
          id: rowColumn
          width: parent.width
          spacing: Style.space(3)
          Repeater {
            model: root.panelRows
            delegate: Column {
              id: repoRow
              required property var modelData
              width: rowColumn.width
              spacing: Style.space(3)
              // Элемент списка (кликабельный во вкладке Branches)
              MouseArea {
                width: parent.width
                height: Style.space(32)
                enabled: !root.busy
                Rectangle {
                  anchors.fill: parent
                  color: parent.pressed ?
Util.alpha(Color.foreground, 0.1) : (parent.containsMouse ?
Util.alpha(Color.foreground, 0.05) : "transparent")
```

```
                radius: Style.cornerRadius
              }
              Row {
                anchors.fill: parent
                anchors.leftMargin: Style.space(6)
                anchors.rightMargin: Style.space(6)
                spacing: Style.space(6)
                Text {
                  width: parent.width - stateLabel.width -
parent.spacing
                  anchors.verticalCenter: parent.verticalCenter
                  // Во вкладке "branches" пишем "Репозиторий /
ветка", иначе стандартный dest
                  text: modelData.dest
                  color: Color.foreground
                  font.family: Style.font.family
                  font.pixelSize: Style.font.caption
                  font.bold: root.panelAction === "branches"
                  elide: Text.ElideRight
                }
                Text {
                  id: stateLabel
                  width: Math.min(implicitWidth, parent.width *
0.4)
                  anchors.verticalCenter: parent.verticalCenter
                  text: modelData.label
                  color: root.rowColor(modelData.state)
                  font.family: Style.font.family
                  font.pixelSize: Style.font.caption
                  horizontalAlignment: Text.AlignRight
                  elide: Text.ElideRight
                }
              }
              onClicked: {
                if (root.panelAction === "branches") {
                  root.selectedRepoData = modelData
                  root.newBranchName = ""
                  root.overlayMessage = ""
                  root.overlayError = ""

                  // Сначала принудительно выставляем значения
для элементов управления в оверлее,
                  // чтобы Dropdown не залипал на старых данных
                  overlayBranchPicker.options =
modelData.branches || []
                  overlayBranchPicker.value = modelData.branch
```

```
|| ""

                    root.branchOverlayOpen = true

                    Qt.callLater(function() {
                      overlayNewBranchField.forceActiveFocus();
                    });
                  }
                }
              }
              Text {
                width: parent.width
                visible: root.panelAction !== "branches" &&
modelData.detail !== undefined && modelData.detail !== ""
                text: modelData.detail
                color: root.rowDetailColor(modelData.state)
                font.family: Style.font.family
                font.pixelSize: Style.font.caption * 0.9
                wrapMode: Text.Wrap
                maximumLineCount: 3
                elide: Text.ElideRight
              }
            }
          }
        }
      }
    }
    // Оверлей для управления ветками конкретного репозитория
    // Оверлей для управления ветками конкретного репозитория
    Rectangle {
      id: branchOverlay
      anchors.fill: parent
      visible: root.branchOverlayOpen
      color: Color.popups.background || "#242424"
      radius: Style.cornerRadius
      z: 5

      // Перехватываем фокус при открытии оверлея
      onVisibleChanged: {
          if (visible) {
              Qt.callLater(function() {
                  overlayNewBranchField.forceActiveFocus();
              });
          }
      }
```

```
      Column {
        anchors.fill: parent
        anchors.margins: Style.space(12)
        spacing: Style.space(10)

        Row {
          width: parent.width
          spacing: Style.space(6)

          Text {
            width: parent.width - closeOverlayButton.width -
fetchBranchButton.width - (parent.spacing * 2)
            text: root.selectedRepoData ? "⚙️ " +
root.selectedRepoData.dest : "Branch management"
            color: Color.foreground
            font.family: Style.font.family
            font.pixelSize: Style.font.body
            font.bold: true
            elide: Text.ElideRight
            anchors.verticalCenter: parent.verticalCenter
          }

          // Кнопка Fetch в оверлее
          Button {
            id: fetchBranchButton
            text: root.busy && root.panelAction === "fetch" ? "⏳
Fetching…" : "🔄 Fetch"
            fontSize: Style.font.caption
            bordered: true
            enabled: !root.busy && root.selectedRepoData &&
root.selectedRepoData.state !== "missing"
            onClicked: {
              if (root.selectedRepoData) {
                root.runFetch(root.selectedRepoData.dest)
              }
            }
          }

          Button {
            id: closeOverlayButton
            text: "×"
            fontSize: Style.font.caption
            onClicked: {
              root.branchOverlayOpen = false
              root.overlayMessage = ""
              root.overlayError = ""
```

```
            }
          }
        }

        // Индикатор недоступности переключения веток
        Text {
          width: parent.width
          visible: root.selectedRepoData && (!
root.selectedRepoData.eligible || root.selectedRepoData.state ===
"missing")
          text: "Branch switch unavailable: " +
(root.selectedRepoData ? (root.selectedRepoData.ineligibleReason
|| "Repository must be clean") : "")
          color: Color.urgent
          font.family: Style.font.family
          font.pixelSize: Style.font.caption
          wrapMode: Text.Wrap
        }

        // Сообщение о результате Fetch (оверлея перекрывает общий
summary/error панели)
        Text {
          width: parent.width
          visible: root.overlayError !== "" ||
root.overlayMessage !== ""
          text: root.overlayError !== "" ? root.overlayError :
root.overlayMessage
          color: root.overlayError !== "" ? Color.urgent :
Color.accent
          font.family: Style.font.family
          font.pixelSize: Style.font.caption
          wrapMode: Text.Wrap
        }

        // Элементы управления (активны, если переключение
разрешено)
        Column {
          width: parent.width
          spacing: Style.space(8)
          visible: root.selectedRepoData &&
root.selectedRepoData.eligible && root.selectedRepoData.state !==
"missing"

          Row {
            width: parent.width
            spacing: Style.space(6)
```

```
            Dropdown {
              id: overlayBranchPicker
              width: parent.width - overlaySwitchButton.width -
parent.spacing
              options: root.selectedRepoData ?
(root.selectedRepoData.branches || []) : []
              value: root.selectedRepoData ?
root.selectedRepoData.branch : ""
              showLabel: false
              fontFamily: Style.font.family
            }

            Button {
              id: overlaySwitchButton
              text: "Switch"
              bordered: true
              fontSize: Style.font.caption
              enabled: overlayBranchPicker.value &&
overlayBranchPicker.value !== (root.selectedRepoData ?
root.selectedRepoData.branch : "")
              onClicked:
root.switchBranch(root.selectedRepoData.dest,
overlayBranchPicker.value, false)
            }
          }

          Rectangle {
            width: parent.width
            height: 1
            color: Color.foreground
            opacity: 0.1
          }

          Row {
            width: parent.width
            spacing: Style.space(6)

            TextField {
              id: overlayNewBranchField
              width: parent.width - overlayCreateButton.width -
parent.spacing
              placeholderText: "new branch name"
              font.pixelSize: Style.font.caption
              text: root.newBranchName
```

```
              // Включаем фокус на уровне движка QML
              activeFocusOnTab: true

              // Перехватываем клик мыши на самом глубоком уровне
              MouseArea {
                anchors.fill: parent
                propagateComposedEvents: false // Блокируем кражу
фокуса родительскими элементами

                onPressed: (mouse) => {
                  overlayNewBranchField.forceActiveFocus();
                  mouse.accepted = false; // Передаем событие
клика самому TextField, чтобы появился курсор
                }
              }

              onTextChanged: root.newBranchName = text
              onAccepted: if (text.trim() !== "")
root.switchBranch(root.selectedRepoData.dest, text.trim(), true)
            }

            Button {
              id: overlayCreateButton
              text: "Create"
              bordered: true
              fontSize: Style.font.caption
              enabled: root.newBranchName.trim() !== ""
              onClicked:
root.switchBranch(root.selectedRepoData.dest,
root.newBranchName.trim(), true)
            }
          }
        }
      }
    }
    ConfirmDialog {
      id: vpnConfirm
      anchors.fill: parent
      z: 10
      opened: root.vpnConfirmOpen
      message: "VPN " + root.vpnConnection + " is off. Turn it on
and continue with " + root.actionLabel(root.pendingAction) + "?"
      cancelText: "Cancel"
      confirmText: "Turn on"
      background: Color.popups.background
      foreground: Color.popups.text
```

```
        scrim: Util.alpha(Color.background, 0.7)
        selectedBackground: Util.alpha(Color.foreground, 0.08)
        selectedText: Color.accent
        fontFamily: Style.font.family
        cornerRadius: Style.cornerRadius
        onCanceled: root.cancelVpnConfirm()
        onConfirmed: root.connectVpn()
    }
  }
}
```

File: gen-pdf.sh

```
#! /bin/bash

repomix --style markdown
pandoc repomix-output.md -o output.pdf --pdf-engine=weasyprint
rm repomix-output.md
```

File: manifest.json

```
{
  "schemaVersion": 1,
  "id": "kai.git-status",
  "name": "BCAD workspace",
  "version": "2.0.0",
  "author": "kai",
  "description": "Shows status of the BCAD multi-repo workspace
        and provides status, branches, pull, and push actions",
  "kinds": [
    "bar-widget"
  ],
  "entryPoints": {
    "barWidget": "BarWidget.qml"
  },
  "barWidget": {
    "displayName": "BCAD workspace",
    "description": "Status, branches, pull, and push across
        workspace packages",
    "category": "Development",
    "allowMultiple": false,
    "defaultSection": "right",
    "defaults": {
      "workspacePath": "~/BAS/bcad-ws",
      "vpnMode": "auto",
```

```
      "vpnConnection": "FALT"
    },
    "schema": [
      {
        "key": "workspacePath",
        "type": "string",
        "label": "Workspace path",
        "description": "Directory containing packages.json and
        packages/"
      },
      {
        "key": "vpnMode",
        "type": "enum",
        "options": ["auto", "always", "never"],
        "defaultValue": "auto",
        "label": "VPN mode",
        "description": "auto: require the VPN only when a profile
        named vpnConnection exists on this machine (work machines
        without it pull/push directly); always: always check and
        offer to connect; never: skip VPN handling entirely"
      },
      {
        "key": "vpnConnection",
        "type": "string",
        "defaultValue": "FALT",
        "label": "VPN connection",
        "description": "NetworkManager connection name used for
        pull/push; empty disables VPN handling"
      }
    ]
  }
}
```

File: workspace.py

```
#!/usr/bin/env python3
import json
import os
import subprocess
import sys

DEFAULT_WORKSPACE = os.path.expanduser("~/BAS/bcad-ws")
DEFAULT_BRANCH = "main"
GIT_TIMEOUT = 10
NETWORK_TIMEOUT = 180
```

```
def git_env():
    env = os.environ.copy()
    env["GIT_TERMINAL_PROMPT"] = "0"
    env["GIT_EDITOR"] = "true"
    env["GIT_SEQUENCE_EDITOR"] = "true"
    env.setdefault("GIT_SSH_COMMAND", "ssh -o BatchMode=yes")
    return env


def run_git(path, args, timeout=GIT_TIMEOUT):
    try:
        proc = subprocess.run(
            ["git", "-C", path] + args,
            capture_output=True,
            text=True,
            timeout=timeout,
            env=git_env(),
        )
        return proc.returncode, proc.stdout.strip(),
        proc.stderr.strip()
    except subprocess.TimeoutExpired:
        return 124, "", "timed out"
    except (FileNotFoundError, OSError) as exc:
        return 1, "", str(exc)


def read_repos(workspace):
    config_path = os.path.join(workspace, "packages.json")
    with open(config_path, encoding="utf-8") as handle:
        data = json.load(handle)

    if not isinstance(data, list):
        raise ValueError("packages.json must be an array")

    repos = []
    for item in data:
        if not isinstance(item, dict):
            continue
        dest = str(item.get("dest") or "").strip()
        if not dest:
            continue
        repos.append(
            {
                "url": str(item.get("url") or ""),
                "dest": dest,
```

```
                "branch": str(item.get("branch") or
        DEFAULT_BRANCH),
            }
        )
    return repos


def repo_path(workspace, dest):
    return os.path.join(workspace, "packages", dest)


def inspect_repo(path):
    row = {
        "branch": "",
        "currentRef": "",
        "dirty": 0,
        "state": "",
        "label": "",
        "detail": "",
    }

    if not os.path.isdir(path):
        row.update(state="missing", label="Missing",
        detail="package directory not found")
        return row

    code, out, err = run_git(path, ["rev-parse", "--is-inside-
        work-tree"])
    if code != 0 or out != "true":
        row.update(
            state="error",
            label="Not a git repository",
            detail=err or "not a git repository",
        )
        return row

    code, out, _ = run_git(path, ["symbolic-ref", "-q", "HEAD"])
    if code == 0 and out:
        row["currentRef"] = out
        prefix = "refs/heads/"
        row["branch"] = out[len(prefix):] if
        out.startswith(prefix) else out
    else:
        code, out, _ = run_git(path, ["rev-parse", "--short",
        "HEAD"])
        row["branch"] = out or "(detached)"
    return row
```

```
def working_tree_clean(path):
    code, out, _ = run_git(path, ["status", "--porcelain", "--
        untracked-files=normal"])
    if code != 0:
        return False, "git status failed"

    lines = [line for line in out.splitlines() if line.strip()]
    if not lines:
        return True, ""

    untracked = any(line.startswith("??") for line in lines)
    tracked = any(not line.startswith("??") for line in lines)
    if tracked and untracked:
        return False, "modified + untracked files"
    if untracked:
        return False, "untracked files"
    return False, "modified files"


def all_branches(path):
    # Получаем локальные и удаленные ветки одной командой
    code, out, _ = run_git(
        path,
        ["branch", "-a", "--format=%(refname:short)"],
    )
    if code != 0:
        return []
    branches = []
    for line in out.splitlines():
        name = line.strip()
        if not name or "->" in name:
            continue
        # Убираем префикс "origin/", чтобы в выпадающем списке
        имена были чистыми
        if name.startswith("origin/"):
            short_name = name[7:]
            if short_name not in branches:
                branches.append(short_name)
        else:
            if name not in branches:
                branches.append(name)
    return sorted(branches)
```

```
def action_status(workspace, repos):
    rows = []
    dirty = untracked_count = missing = errors = 0

    for repo in repos:
        path = repo_path(workspace, repo["dest"])
        row = inspect_repo(path)
        row["dest"] = repo["dest"]
        row["expectedBranch"] = repo["branch"]

        if row["state"] == "missing":
            missing += 1
        elif row["state"] == "error":
            errors += 1
        else:
            code, out, _ = run_git(
                path, ["status", "--porcelain", "--untracked-
files=normal"]
            )
            if code != 0:
                row.update(state="error", label="Error",
detail="git status failed")
                errors += 1
            else:
                lines = [line for line in out.splitlines() if
line.strip()]
                untracked_lines = [line for line in lines if
line.startswith("??")]
                untracked = len(untracked_lines)
                tracked = len(lines) - untracked
                row["dirty"] = tracked
                row["untracked"] = untracked

                if tracked > 0:
                    label = "Modified ({})".format(tracked)
                    if untracked > 0:
                        label += " · {}
untracked".format(untracked)
                    row.update(state="modified", label=label)
                    dirty += 1
                elif untracked > 0:
                    names = [line[3:].strip() for line in
untracked_lines[:3]]
                    detail = "untracked: " + ", ".join(names)
                    if untracked > len(names):
                        detail += " …"
                    row.update(
                        state="untracked",
```

```
                    label="Up to date · {}
        untracked".format(untracked),
                    detail=detail,
                )
                untracked_count += 1
            else:
                row.update(state="clean", label="Up to date")

        rows.append(row)

    parts = []
    if dirty:
        parts.append("{} modified".format(dirty))
    if untracked_count:
        parts.append("{} untracked".format(untracked_count))
    if missing:
        parts.append("{} missing".format(missing))
    if errors:
        parts.append("{} errors".format(errors))
    clean = len(repos) - dirty - untracked_count - missing -
        errors
    if clean > 0:
        parts.append("{} up to date".format(clean))

    return {
        "ok": dirty == 0 and untracked_count == 0 and missing ==
        0 and errors == 0,
        "summary": " · ".join(parts) if parts else "No
        repositories",
        "dirtyCount": dirty,
        "untrackedCount": untracked_count,
        "missingCount": missing,
        "errorCount": errors,
        "rows": rows,
    }


def action_branches(workspace, repos):
    rows = []
    mismatched = missing = errors = 0
    for repo in repos:
        path = repo_path(workspace, repo["dest"])
        row = inspect_repo(path)
        row["dest"] = repo["dest"]
        row["expectedBranch"] = repo["branch"]
        row["eligible"] = False
        row["ineligibleReason"] = ""
```

```
    row["branches"] = []
    if row["state"] == "missing":
        missing += 1
        row["ineligibleReason"] = "repository unavailable"
    elif row["state"] == "error":
        errors += 1
        row["ineligibleReason"] = "repository unavailable"
    else:
        clean, reason = working_tree_clean(path)
        row["eligible"] = clean
        if not clean:
            row["ineligibleReason"] = reason
        row["branches"] = all_branches(path)

        # Гарантируем, что текущая ветка записана в
    row["branch"]
        if not row.get("branch"):
            code, out, _ = run_git(path, ["symbolic-ref", "--
    short", "-q", "HEAD"])
            row["branch"] = out.strip() if code == 0 else
    "detached"

        if row["branch"] != repo["branch"]:
            row.update(
                state="mismatch",
                label=row["branch"],
                detail="expected {}".format(repo["branch"]),
            )
            mismatched += 1
        else:
            row.update(state="ok", label=row["branch"])
    rows.append(row)

parts = []
if mismatched:
    parts.append("{} on unexpected branch".format(mismatched))
if missing:
    parts.append("{} missing".format(missing))
if errors:
    parts.append("{} errors".format(errors))
ok = len(repos) - mismatched - missing - errors
if ok > 0:
    parts.append("{} on expected branch".format(ok))
return {
    "ok": mismatched == 0 and missing == 0 and errors == 0,
    "summary": " · ".join(parts) if parts else "No
    repositories",
```

```
        "mismatchCount": mismatched,
        "missingCount": missing,
        "errorCount": errors,
        "rows": rows,
    }

def switch_branch_error(workspace, repos, message):
    result = action_branches(workspace, repos)
    result["action"] = "switch-branch"
    result["error"] = message
    result["summary"] = ""
    return result


def action_switch_branch(workspace, repos, dest, branch, create):
    branch = (branch or "").strip()
    target = next((repo for repo in repos if repo["dest"] ==
        dest), None)
    if target is None:
        return switch_branch_error(workspace, repos, "repository
        not found: " + dest)
    if not branch:
        return switch_branch_error(workspace, repos, "branch name
        is required")

    path = repo_path(workspace, dest)
    row = inspect_repo(path)
    if row["state"] in ("missing", "error"):
        return switch_branch_error(workspace, repos, "repository
        unavailable: " + dest)

    clean, reason = working_tree_clean(path)
    if not clean:
        return switch_branch_error(
            workspace, repos, "repository is not clean: " + reason
        )

    code, _, _ = run_git(path, ["check-ref-format", "--branch",
        branch])
    if code != 0:
        return switch_branch_error(workspace, repos, "invalid
        branch name: " + branch)

    code, _, err = run_git(
        path, ["show-ref", "--verify", "--quiet", "refs/heads/" +
        branch]
    )
    exists = code == 0
```

```
if code not in (0, 1):
    return switch_branch_error(
        workspace, repos, "could not check branch: " + (err
    or "git show-ref failed")
    )

if create:
    if exists:
        return switch_branch_error(
            workspace, repos, "branch already exists: " +
    branch
        )
    code, _, err = run_git(path, ["switch", "-c", branch])
    if code != 0:
        code, _, err = run_git(path, ["checkout", "-b",
    branch])
    if code != 0:
        return switch_branch_error(
            workspace, repos, err or "could not create branch"
        )
    notice = "Created {} in {}".format(branch, dest)
else:
    # 1. Проверяем, существует ли ветка локально
    code_local, _, _ = run_git(path, ["show-ref", "--verify",
    "--quiet", "refs/heads/" + branch])

    if code_local == 0:
        # Локальная ветка есть — переключаемся стандартно
        code, _, err = run_git(path, ["switch", branch])
        if code != 0:
            code, _, err = run_git(path, ["checkout", branch])
    else:
        # 2. Локальной ветки нет. Проверяем, есть ли она на
    origin
        code_remote, _, _ = run_git(path, ["show-ref", "--
    verify", "--quiet", "refs/remotes/origin/" + branch])

        if code_remote == 0:

    # Ветка есть на origin — создаем локальную с явным
    трекингом удаленной
            code, _, err = run_git(path, ["switch", "-c",
    branch, "--track", "origin/" + branch])
            if code != 0:
                code, _, err = run_git(path, ["checkout", "-
    b", branch, "--track", "origin/" + branch])
        else:
            # 3. Ветки нет ни локально, ни на origin
    (например, она на другом remote)
```

```
                # Пробуем обычный switch/checkout — вдруг git сам
        разберется по своим механизмам
                code, _, err = run_git(path, ["switch", branch])
                if code != 0:
                    code, _, err = run_git(path, ["checkout",
        branch])

            if code != 0:
                return switch_branch_error(
                    workspace, repos, err or "could not switch branch"
                )
            notice = "Switched {} to {}".format(dest, branch)

        result = action_branches(workspace, repos)
        summary = notice
        if result.get("summary"):
            summary += " · " + result["summary"]
        result["action"] = "switch-branch"
        result["summary"] = summary
        result["dest"] = dest
        result["branch"] = branch
        return result


    def action_fetch(workspace, repos, dest):
        target = next((repo for repo in repos if repo["dest"] ==
            dest), None)
        if target is None:
            return {"ok": False, "error": "Repository not found"}
        path = repo_path(workspace, dest)
        code, out, err = run_git(path, ["fetch", "--all"],
            timeout=NETWORK_TIMEOUT)
        result = action_branches(workspace, repos)
        result["action"] = "branches"
        if code != 0:
            result["error"] = err or "git fetch failed"
        else:
            result["summary"] = "Fetched updates for " + dest
        return result


    def action_network(workspace, repos, action):
        rows = []
        ok_count = failed = missing = 0

        for repo in repos:
            path = repo_path(workspace, repo["dest"])
```

```
        row = inspect_repo(path)
        row["dest"] = repo["dest"]
        row["expectedBranch"] = repo["branch"]

        if row["state"] in ("missing", "error"):
            if row["state"] == "missing":
                missing += 1
            else:
                failed += 1
            row["state"] = "failed"
            rows.append(row)
            continue

        branch = row["branch"]
        if branch in ("", "(detached)"):
            row.update(
                state="failed",
                label="detached HEAD",
                detail="cannot determine current branch",
            )
            failed += 1
            rows.append(row)
            continue

        args = ["push", "origin", branch] if action == "push"
         else ["pull", "--no-edit", "origin", branch]
        code, out, err = run_git(path, args,
         timeout=NETWORK_TIMEOUT)
        if code == 0:
            row.update(state="ok", label="{} -
         Ok".format(branch), detail=out)
            ok_count += 1
        else:
            row.update(
                state="failed",
                label="{} - Failed".format(branch),
                detail=err or out or "git command failed",
            )
            failed += 1

        rows.append(row)

    parts = []
    if ok_count:
        parts.append("{} ok".format(ok_count))
    if failed:
        parts.append("{} failed".format(failed))
```

```
    if missing:
        parts.append("{} missing".format(missing))

    return {
        "ok": failed == 0 and missing == 0,
        "summary": " · ".join(parts) if parts else "No
        repositories",
        "okCount": ok_count,
        "failedCount": failed,
        "missingCount": missing,
        "rows": rows,
    }


def main():
    action = sys.argv[1] if len(sys.argv) > 1 else "status"
    raw_workspace = sys.argv[2] if len(sys.argv) > 2 else
        DEFAULT_WORKSPACE
    workspace =
        os.path.expandvars(os.path.expanduser(raw_workspace))

    result = {
        "action": action,
        "workspace": workspace,
        "configPath": os.path.join(workspace, "packages.json"),
        "ok": False,
        "error": None,
        "summary": "",
        "rows": [],
    }

    try:
        repos = read_repos(workspace)
    except FileNotFoundError:
        result["error"] = "packages.json not found"
        print(json.dumps(result))
        return 0
    except (OSError, ValueError) as exc:
        result["error"] = "invalid packages.json: " + str(exc)
        print(json.dumps(result))
        return 0

    if not repos:
        result["error"] = "packages.json has no repositories"
        print(json.dumps(result))
        return 0
```

```
    if action == "status":
        result.update(action_status(workspace, repos))
    elif action == "branches":
        result.update(action_branches(workspace, repos))
    elif action == "fetch":
        if len(sys.argv) < 4:
            result["error"] = "usage: fetch <workspace> <dest>"
        else:
            result.update(action_fetch(workspace, repos,
        sys.argv[3]))
    elif action == "switch-branch":
        if len(sys.argv) < 6:
            result["error"] = (
                "usage: switch-branch <workspace> <dest> <branch>
        <switch|create>"
            )
        elif sys.argv[5] not in ("switch", "create"):
            result["error"] = "invalid mode: " + sys.argv[5]
        else:
            result.update(
                action_switch_branch(
                    workspace,
                    repos,
                    sys.argv[3],
                    sys.argv[4],
                    sys.argv[5] == "create",
                )
            )
    elif action in ("pull", "push"):
        result.update(action_network(workspace, repos, action))
    else:
        result["error"] = "unknown action: " + action

    print(json.dumps(result))
    return 0


if __name__ == "__main__":
    sys.exit(main())
```